COMPTE RENDU DU TP 1.1 SUR LA DECOUVERTE ET LA CARTOGRAPHIE D'UN RESEAU AVEC NMAP

1. Introduction

Wireshark est un analyseur de paquets réseau permettant de capturer et d'inspecter le trafic en temps réel. Son rôle est fondamental en cybersécurité, car les premières traces d'une intrusion apparaissent généralement au niveau des paquets réseau.

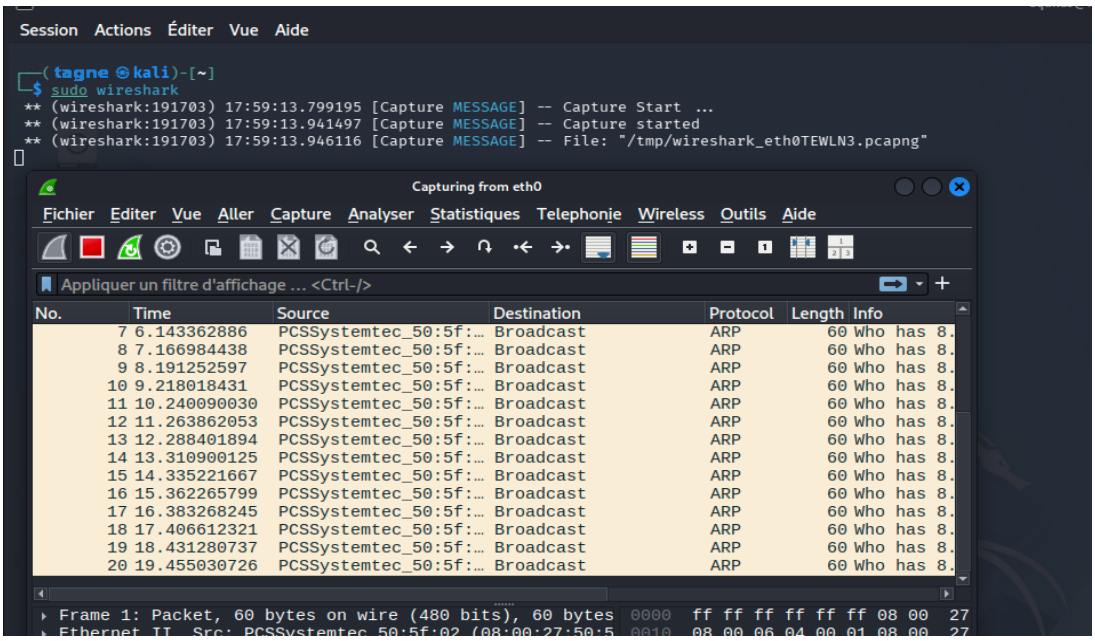
L'analyse du trafic permet ainsi de détecter des comportements anormaux tels que : Les scans de ports, Les tentatives d'intrusion, Les attaques par déni de service, Les activités suspectes sur le réseau.

L'objectif de ce TP est d'observer concrètement le trafic généré par un scan Nmap afin de comprendre les mécanismes utilisés lors d'une phase de reconnaissance réseau.

2. Description des manipulations

Dans un premier temps, la connectivité entre les machines virtuelles a été vérifiée à l'aide de la commande : **ping 192.168.56.10**

Après confirmation de la communication entre les machines, Wireshark a été lancé sur Kali Linux avec la commande : **sudo wireshark**



Une fois la capture démarrée, un scan SYN a été exécuté vers la machine Ubuntu Server à l'aide de la commande suivante : **sudo nmap -sS 192.168.56.10**

L'option **-sS** correspond à un scan SYN (semi-ouvert) qui envoie uniquement des paquets SYN sans finaliser la connexion TCP. Cette méthode rend le scan plus discret.

Après l'exécution du scan, les filtres suivants ont été appliqués dans Wireshark :

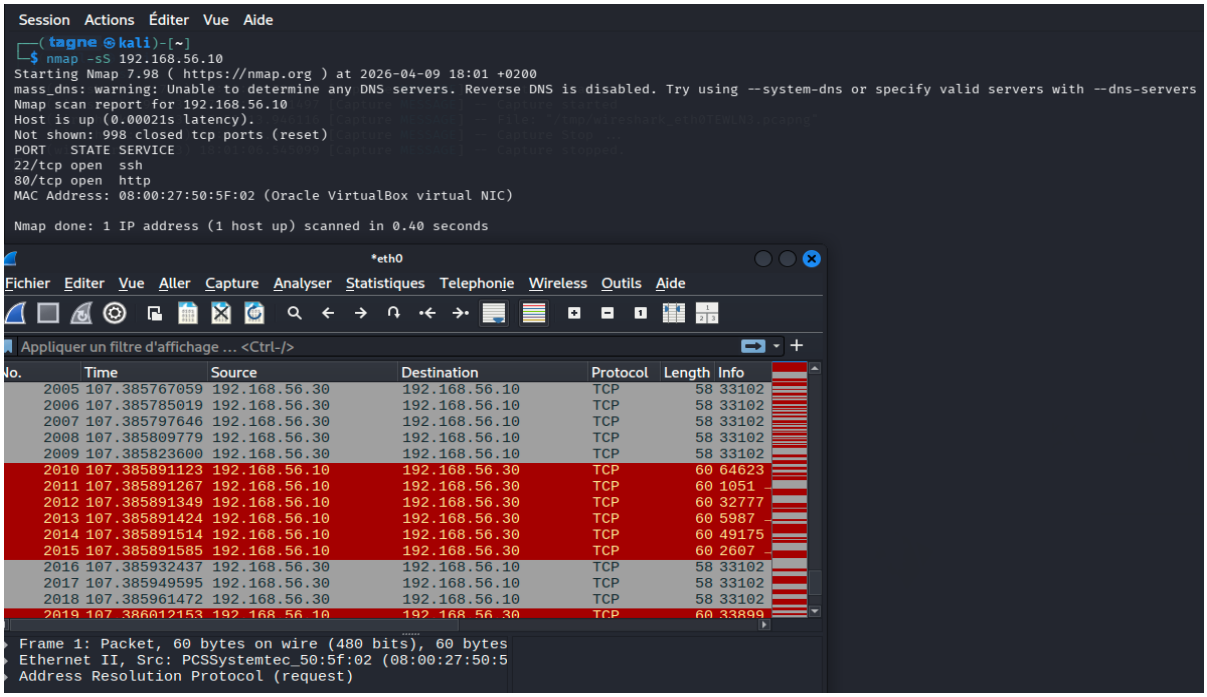
- Filtre SYN : tcp.flags.syn == 1
- Filtre IP cible : ip.addr == 192.168.56.10

Ces filtres ont permis d'isoler les paquets liés au scan.

2. Résultats obtenus avec captures d'écran

Capture 1 : Trafic brut Wireshark

La première capture présente l'exécution de la commande : **nmap -sS 192.168.56.10**

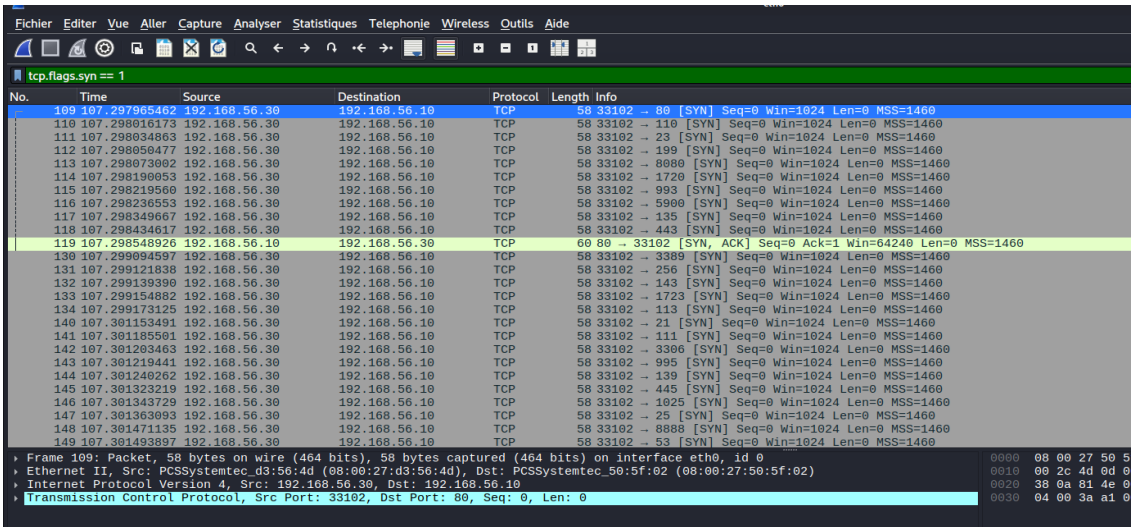


Les ports ouverts détectés sont Port 22 : SSH et Port 80 : http. Cette capture confirme que le scan a été exécuté avec succès. Elle montre une série de paquets envoyés vers la machine cible ubuntu, un grand nombre de paquets capturés en quelques secondes ainsi qu'un grand nombre

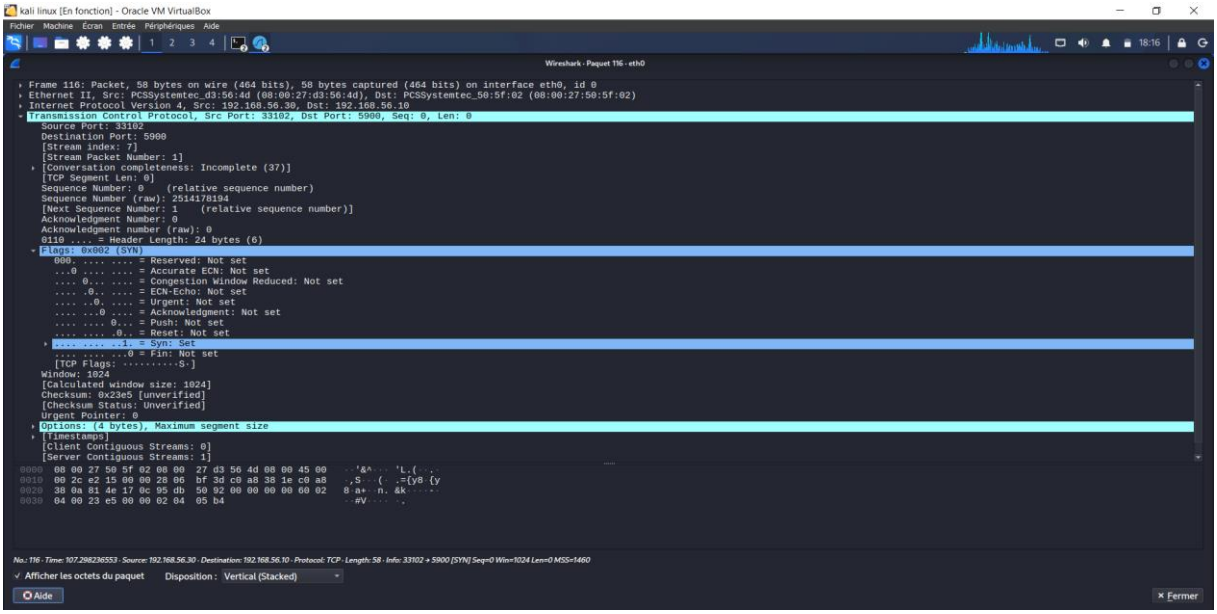
de tentatives de connexions rapide. Ce trafic dense constitue un premier indicateur d'activité anormale, typique d'un scan réseau.

Capture 2 : Filtre SYN

Après application du filtre **: tcp.flags.syn == 1**



On observe : l'envoi de paquets SYN depuis Kali, les réponses SYN-ACK pour ports ouverts, les réponses RST pour ports fermés. Donc on observe uniquement les paquets les paquets contenant le flag SYN qui correspond à la première étape d'une connexion TCP. Cette répétition est caractéristique d'un scan automatisé.



L'analyse détaillée d'un paquet montre :

- Pour port ouvert : SYN → SYN-ACK → RST
- Pour port fermé : SYN → RST

Ces échanges confirment le fonctionnement du scan SYN, et nmap les utilise pour détecter le port. Ainsi, les IDS/IPS utilisent ce principe pour détecter les scans réseaux et identifier les comportements suspects.

4. Analyse technique

Un paquet SYN est un segment TCP dont le flag SYN est activé. Il sert à initier une connexion TCP dans le mécanisme appelé **Three-Way Handshake** : SYN, SYN-ACK, ACK. Il contient un numéro de séquence initial qui permettra d'ordonner les données

Dans le scan SYN **pour détecter l'état d'un port**, on observe la réponse de la machine cible. Si le port est ouvert, elle répond par un SYN-ACK. Si le port est fermé, elle répond immédiatement par un RST.

Un scan réseau se reconnaît par : une rafale rapide de paquets SYN, une même adresse IP cible, des ports qui s'incrémentent rapidement.

Ces caractéristiques permettent donc de détecter un scan réseau ainsi que d'identifier les comportements suspects.

5. Utilisation avancée de Wireshark

- **Filtre IP**

ip.addr == 192.168.56.10 : Permet d'afficher uniquement le trafic d'une machine spécifique. Il est utile pour isoler une cible particulière. Par exemple, lors du scan, il permet d'ignorer le trafic entre autres machines

- **Filtre Port**

tcp.port == 80 : Permet d'analyser uniquement le trafic http donc il sélectionne les paquets dont le port source ou destination est 80 (HTTP). Il sert à surveiller le trafic Web.
Exemple : tcp.port == 80 and http.request (Affiche uniquement les requêtes http).

- **Filtre Reset**

tcp.flags.reset == 1 : Permet d'identifier les connexions refusées. Lors du scan, ce filtre permet d'afficher uniquement les ports fermés.

6. Lien avec la cybersécurité en entreprise

Dans un SOC (Security Operations Center), les analystes utilisent Wireshark pour : Investiguer les des alertes déclenchées par les systèmes de détection automatiques. Ils appliquent des filtres pour confirmer la présence d'un scan réseau, identifier la machine source et les ports ciblés, Analyser les attaques.

Pour détecter rapidement une attaque ils recherchent des signatures comportementales :

- Scan réseau : caractérisé par un grand nombre de paquets SYN envoyés en peu de temps vers plusieurs ports,
- Brute force : caractérisé par de multiples tentatives de connexion
- Attaque DDoS : caractérisé par un trafic massif anormal.

7. Conclusion

Ce TP m'a appris qu'un scan réseau n'est pas invisible : il produit des paquets SYN, SYN-ACK et RST que Wireshark permet d'observer et d'interpréter. Les difficultés rencontrées ont été la synchronisation entre la capture et le scan, ainsi que l'identification précise des flags TCP parmi la masse de paquets. Wireshark présente des limites : il ne capture que le trafic visible par l'interface utilisée, il ne lit pas le contenu des flux chiffrés, et il n'agit pas automatiquement sur les attaques.

Malgré cela, ce TP m'a fait comprendre ce qui se passe réellement sur le réseau et m'a initié au raisonnement d'un analyste SOC